

LIG넥스원 국방 시스템에 변이 특징을 활용한 딥러닝 기반 결함 위치 탐지 적용

양희찬 (지도교수: 김문주)



소프트웨어를 탑재한 국방 무기 체계 시스템

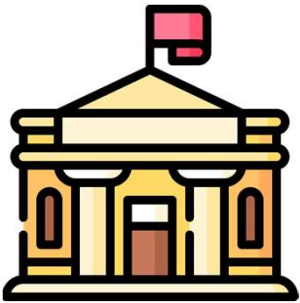
LIG Nex1

국방 무기 체계 시스템



목표: MBFL-based DLFL를 국방 무기체계 시스템에 적용

KAIST



학계

+3 개월 현장 협업

10년 협업을 통한 신뢰 확보

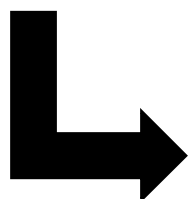


LIG Nex1



산업

국방 무기체계 SW에 MFBL-based DLFL 적용하기 어려운가?



Problems:

- P1** C/C++ 소프트웨어 적용 가능한 **공개 도구 부재**
- P2** 딥러닝 모델 학습 데이터셋 구축의
막대한 실행 비용 (≈ 312 CPU-days)

요약

P1 공개 도구 부재



1. 산업 현장의 C/C++ SW에 적용을 위한 실용적인 MBFL-DLFL 데이터셋 구축 **도구 개발**

P2 막대한 계산 비용



2. 최적화 설정을 통해 나이브(Naïve) 설정 대비 데이터셋 구축 **실행 비용을 74.6% 절감**

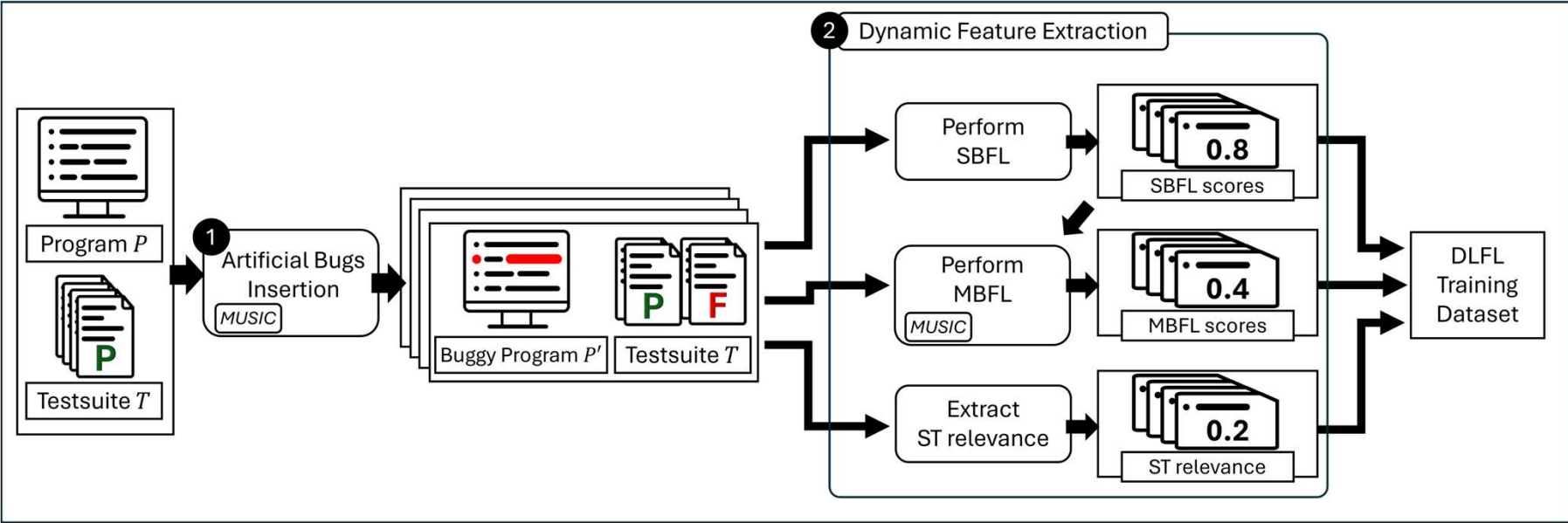


3. LIG넥스원의 무기체계 SW에 MBFL-DLFL 기술 성공적 적용:

- 실행 비용: **74.6% 단축**
- 정확도: Top-1 결함 탐지 **90.97% 달성**

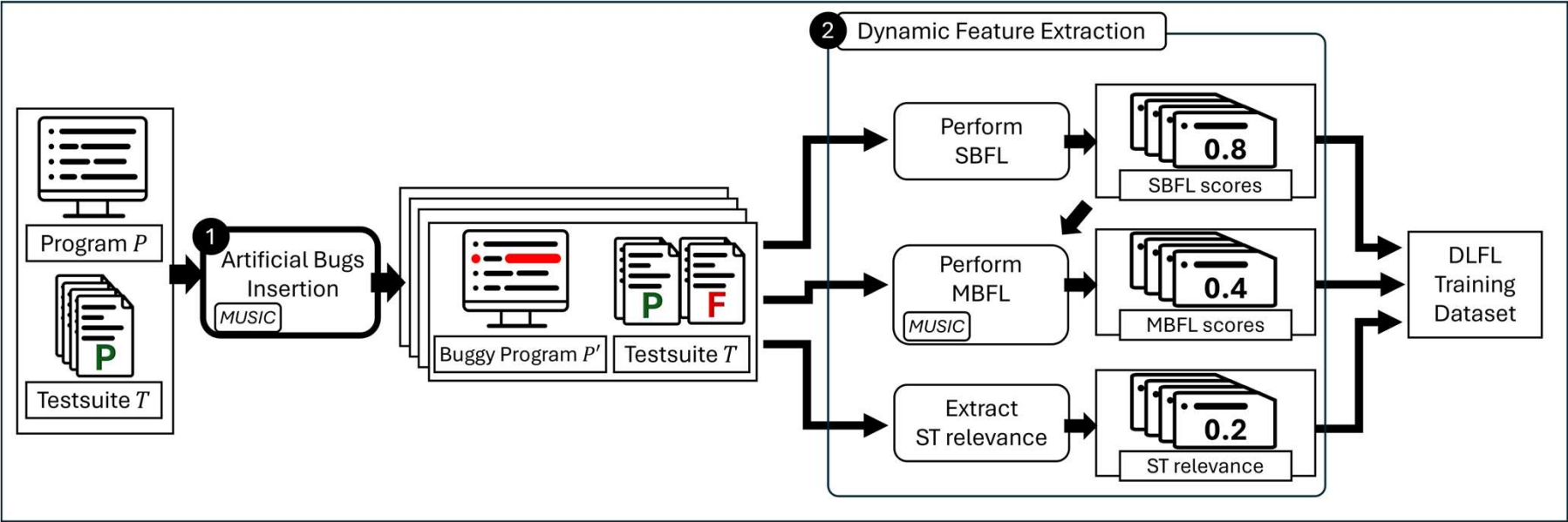
P1 공개 도구의 부재

C/C++ SW 대상 MBFL-DLFL 데이터셋 구축 도구 개발



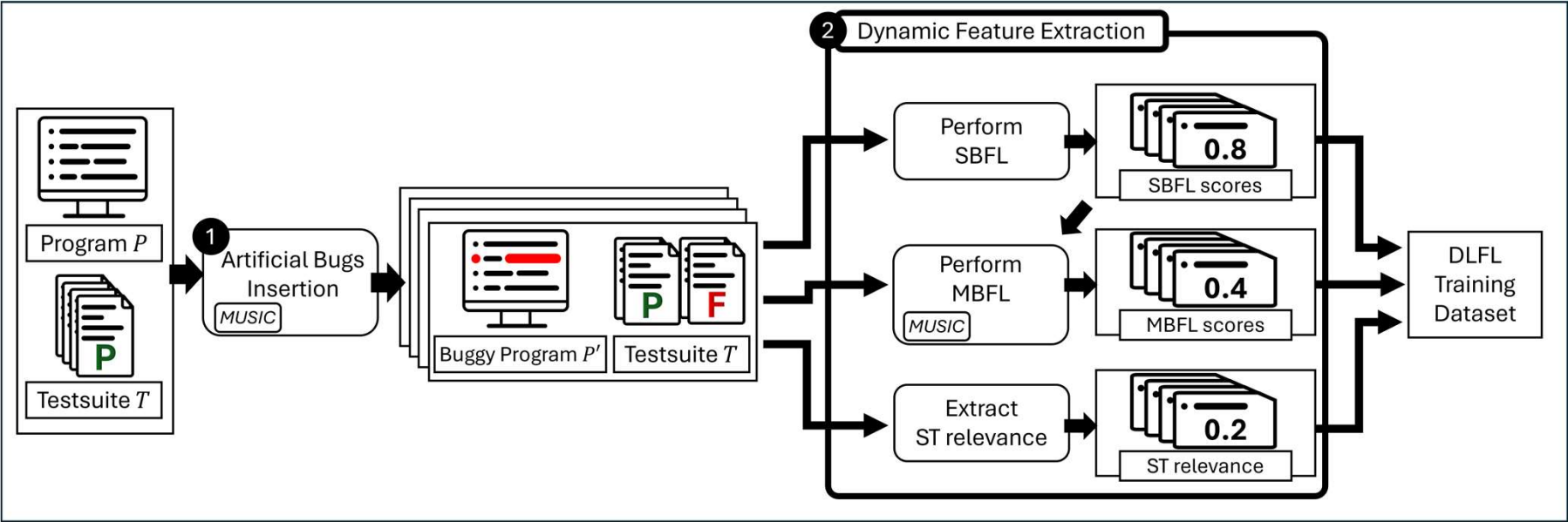
P1 공개 도구의 부재

C/C++ SW 대상 MBFL-DLFL 데이터셋 구축 도구 개발



P1 공개 도구의 부재

C/C++ SW 대상 MBFL-DLFL 데이터셋 구축 도구 개발



P2 데이터셋 구축의 막대한 계산 비용

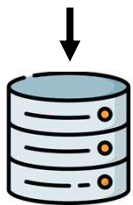
Source Code Lines



Target Line Ratio (r)



Mutant Per Line (m)



Mutant Features

Defects4J 벤치마크(257개 결함) 대상 사전 체계적 실험 진행:

통계적 정확도는 그대로 유지하면서, 실행 비용을 최소화할 수 있는 **최적의 임계값(Threshold)**을 정확히 규명함.

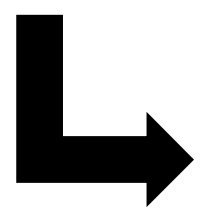
최적의 설정:

대상(r): 의심 부위의 **상위 70% 라인만** 선택

볼륨(m): 선택한 코드 라인당 최대
3개의 변이체 생성

LIG 넥스원 적용

데이커셋 구축 도구
& 최적의 설정 값 ($r=70\%$, $m=3$)



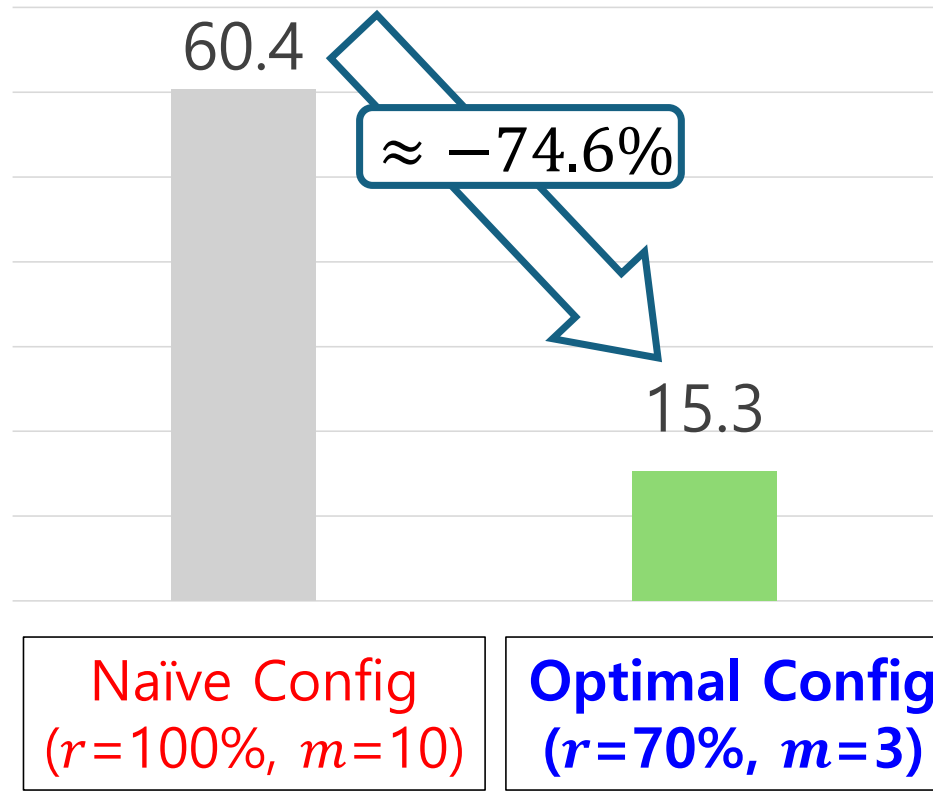
LIG Nex1:
국방 무기체계 SW 적용

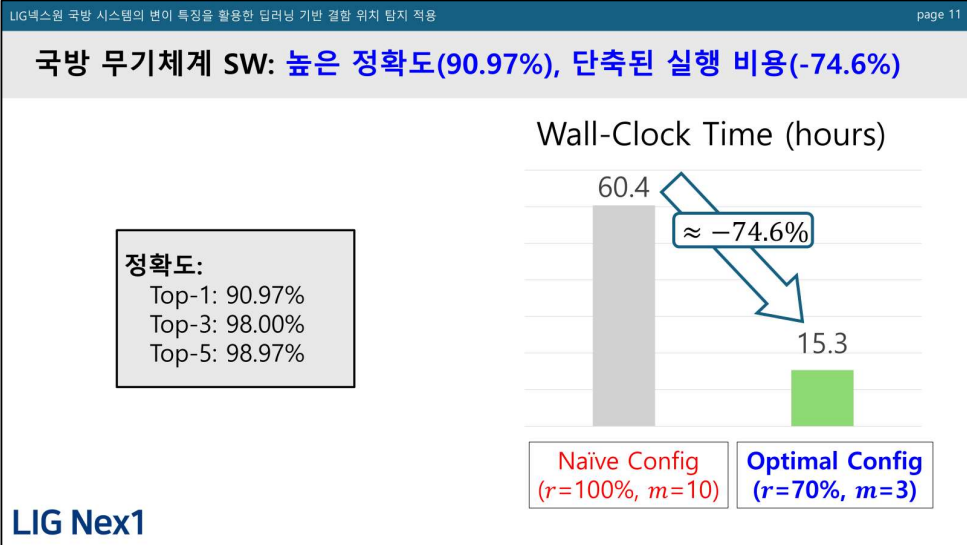
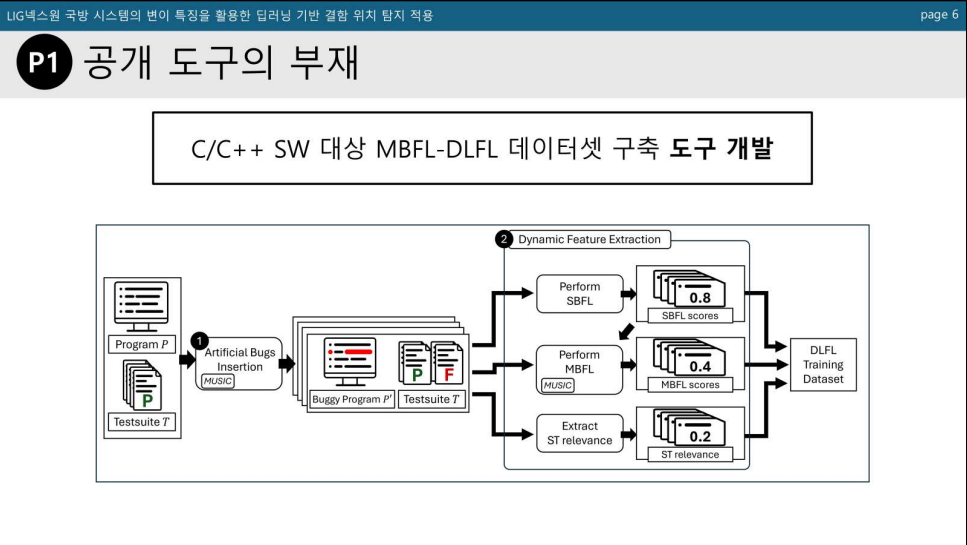
LIG Nex1

국방 무기체계 SW: 높은 정확도(90.97%), 단축된 실행 비용(-74.6%)

정확도:
Top-1: 90.97%
Top-3: 98.00%
Top-5: 98.97%

Wall-Clock Time (hours)





**LIG 넥스원 국방 무기체계 SW에
MBFL-DLFL 성공적 적용**

Deploying the Framework on Defense SW

LIG Nex1

Subject program: 6 military defense software systems
(deployed in aerospace, maritime, guided weapons)

Evaluation method: 10-fold cross validated repeated 10 times



Subject: 10K+ lines of C/C++ code

Dataset: 300 artificial buggy programs
(due to strict defense security protocols)

Experiment: 10-fold cross-validation
repeated 10 times

P1 공개 도구 부재

Recent works on MBFL-based DLFL:

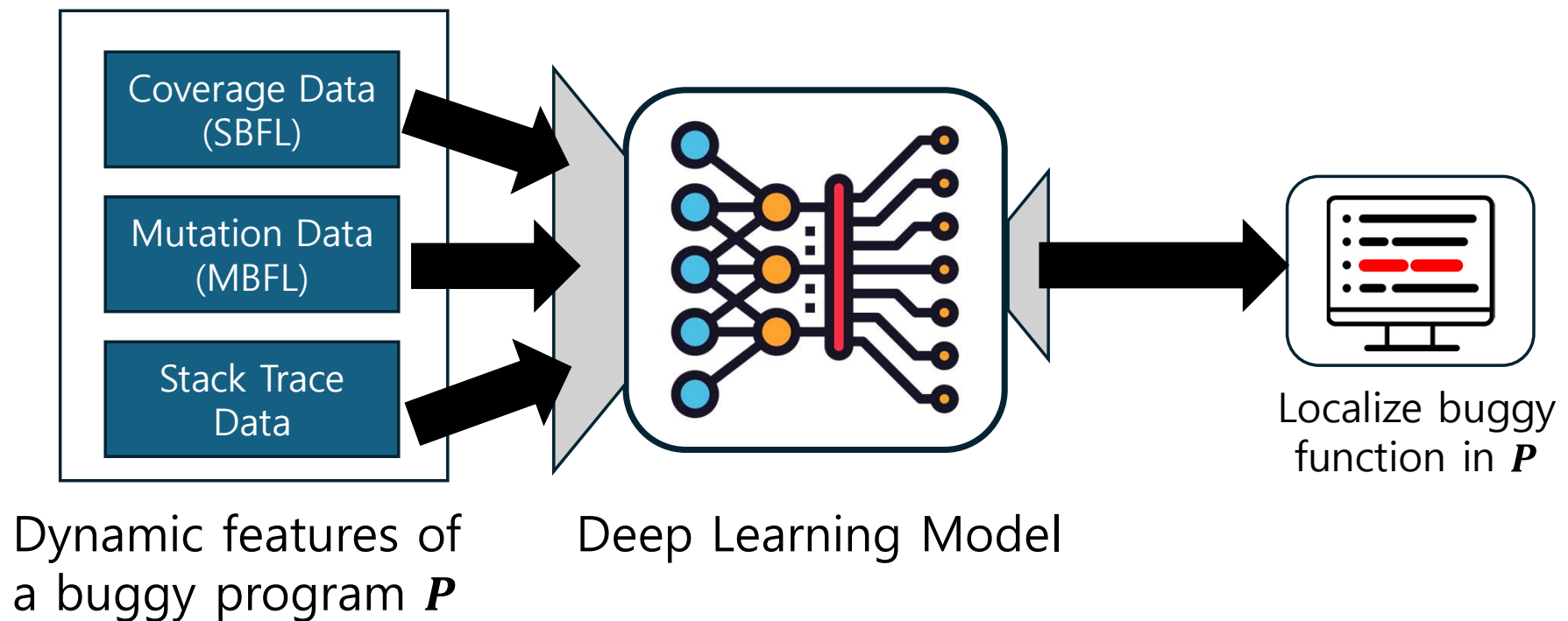
No publicly available MBFL-DLFL framework for C/C++ programs

Paper	Venue	Lang.	Mutation tool	MBFL impl. details
DeepFL	ISSTA '19	Java	PIT-1.15	X
GMBFL	ICSME '23	Java	Not specified	X
MTL-TRANSFER	TOSEM '24	Java	PIT-1.1.5	X
FusionFL	ICST '24	Java	PIT-1.1.5	X
CodeHealer	TOSEM '25	Java	PIT-1.1.5	X
TRANSFER	ICSE '22	Java	PIT-1.1.5	X
ALBFL	IST '21	Java	Not specified	X

Higher Accuracy with Deep Learning Technique

MBFL-based Deep Learning FL (DLFL):

Model learns from **various program features**, showing improved FL accuracy than MBFL



P2 Massive Computation Cost in Extracting Mutant Features

Execute tests mutated programs to **extract mutant features**:

TOO many mutants to test (N) = massive computation cost

$$N = r \times m$$

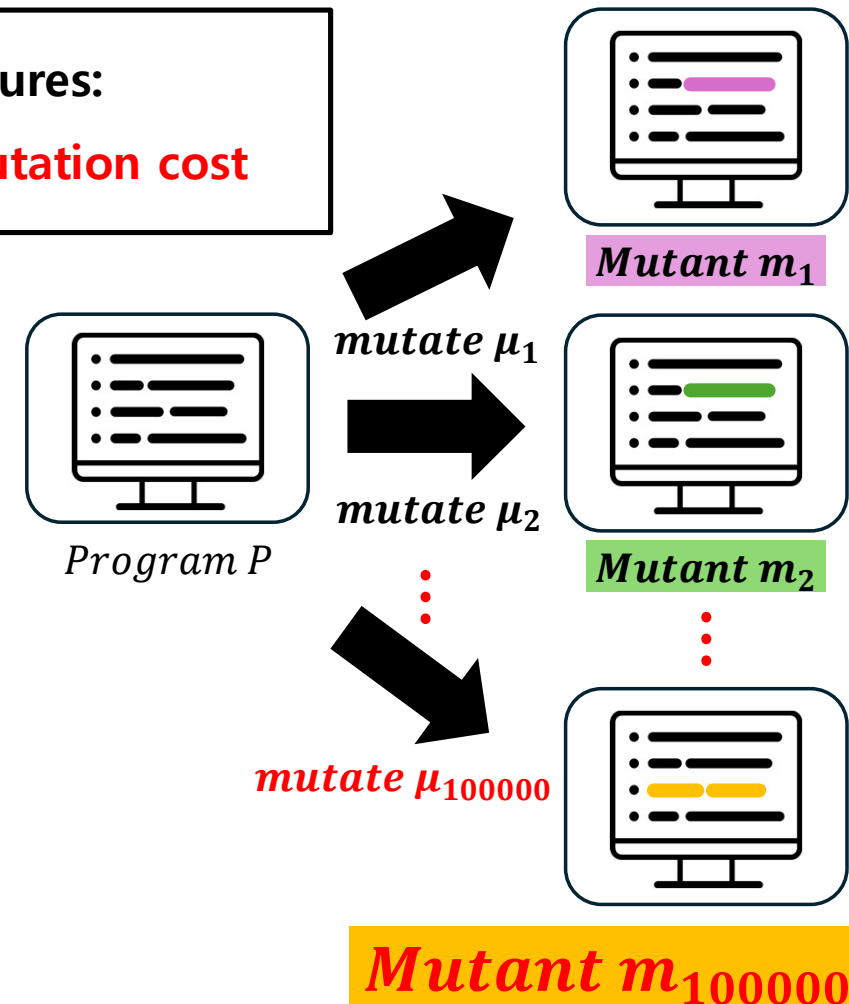
r = # of lines to mutate

m = # mutants per line

Ex. program with 10K lines X 10 mutants per line

= 100K mutants to test

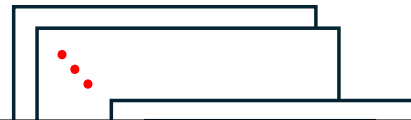
≈ 25 CPU-hours!!!



P2 Massive Computation Cost in Extracting Mutant Features

Weakness of MBFL-DLFL:

constructing the training dataset takes
unfeasible amount of computation cost



Naïve Configuration Cost:

≈ 312 CPU-DAYS!!!

for 300 buggy programs (with 10K line)

Tr

300 buggy programs

From Theoretical Research to Industrial Application



LIG Nex1

" ... Now we can localize a faulty function in a few hours, not in a few days."

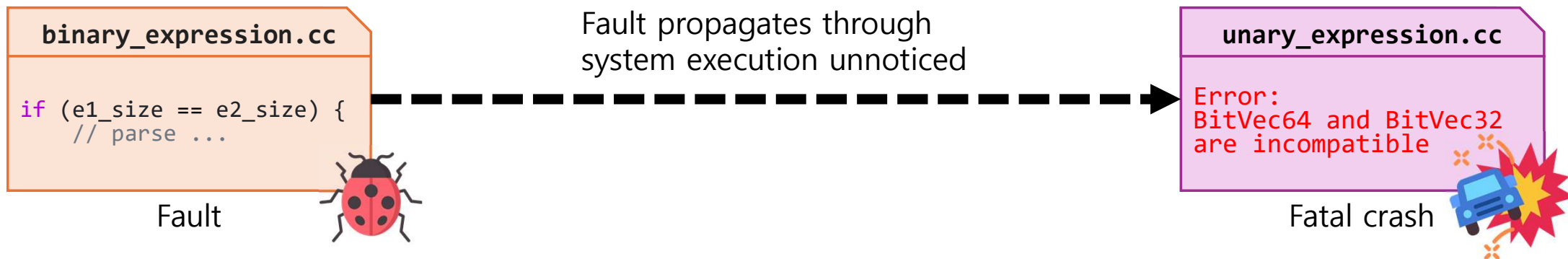
- Lead Researcher, at **LIG Nex1**

The Nightmare of the Distant Bug

Site of system **crash** is rarely the site of actual **fault**

Case Study: CROWN 2.0 **Issue #27**

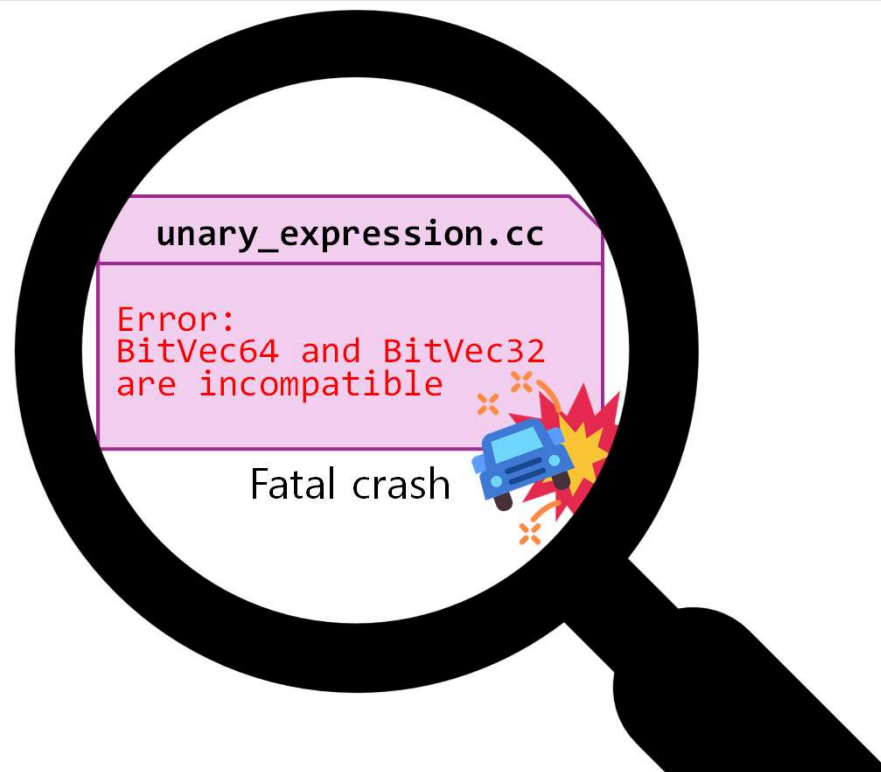
A 32-bit/64-bit shift operator mismatch occurs silently.



Limitation of Traditional Coverage-Based Methods

Coverage-based analysis strictly focuses on the **crash site**

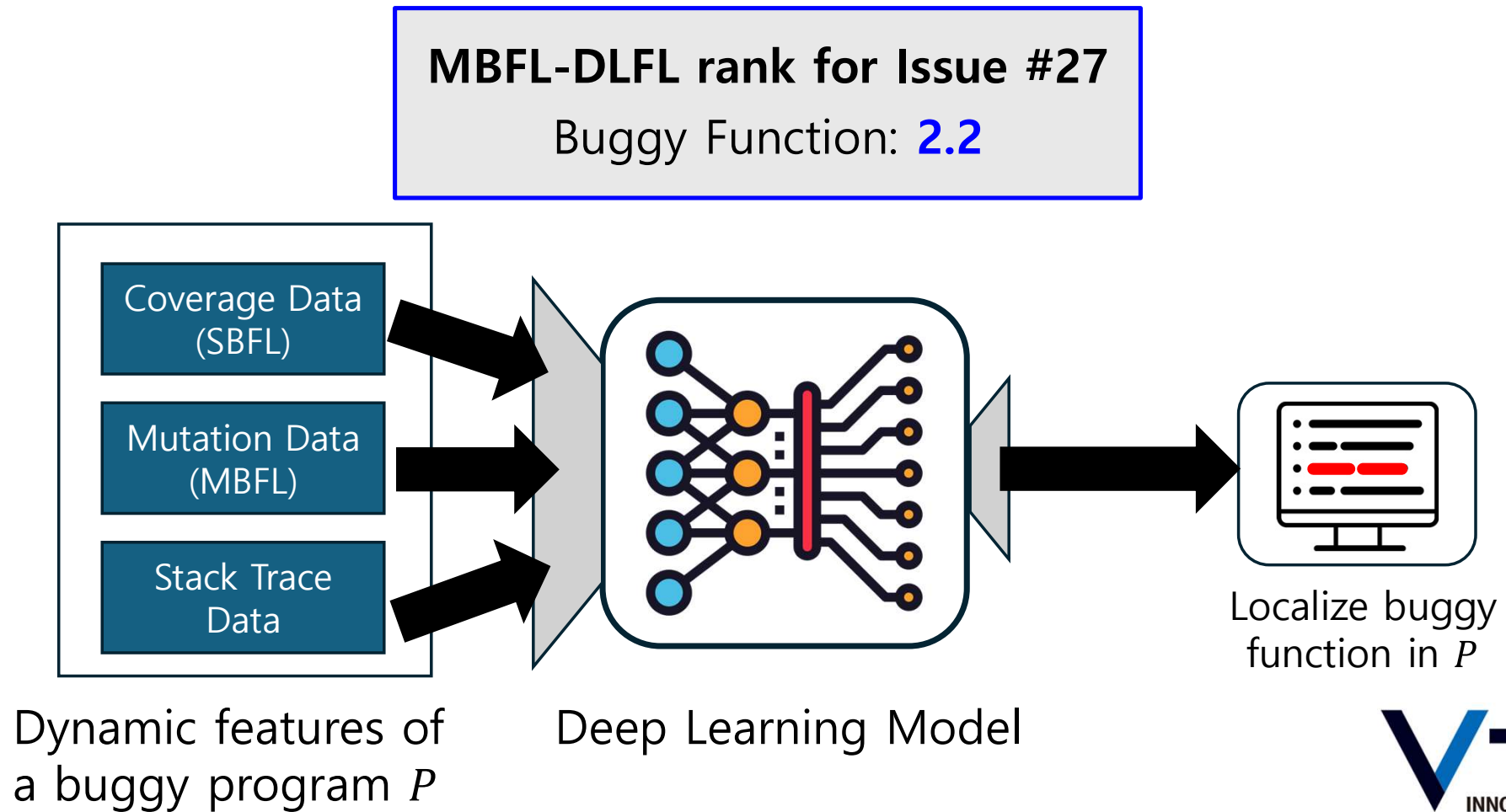
Spectrum-Based Fault Localization (SBFL) relies only on test execution frequencies.



SBFL rank for Issue #27

Buggy Function: **29.0**

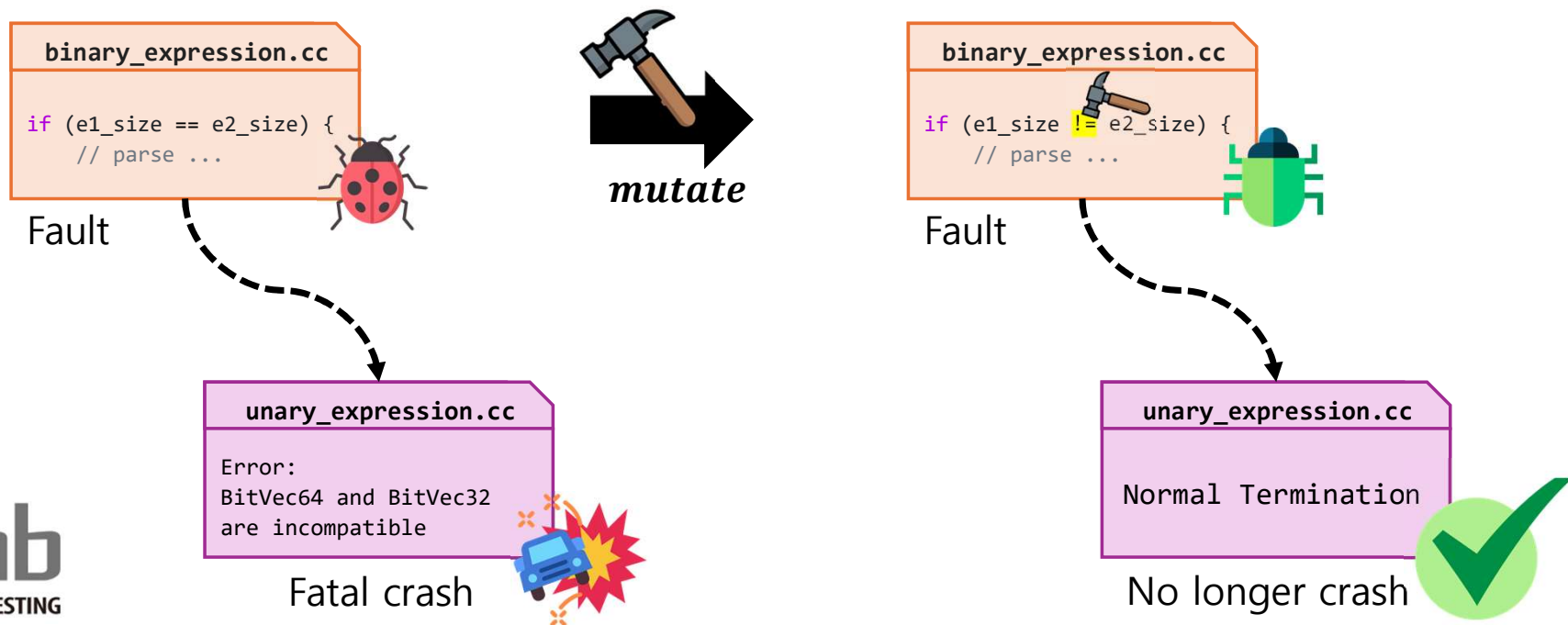
Higher Accuracy with Deep Learning Technique



Strengths of Mutation Features

Mutant signals traces the behavioral impact (crash**) back to its true origin (**fault**)**

Mutation-Based Fault Localization (MBFL) perturbs codes to capture outcome (program behaviors) changes of test cases.



Academia VS Industrial

Focus of existing works in academia:
model architecture over dataset construction efficiency

Feature	Academic Baseline	Our Industrial Framework
Target Language	Java	C/C++
Mutation Engine	PITest (customization required)	MUSIC (customization not-required)
Mutation Volume	Exhaustive (100% of lines / 10 mutants per line)	Optimized (70% of lines / 3 mutants per line)
Processing Time	~7,500 CPU-hours	~1,900 CPU-hours
Tool Availability	Prototypical / Private	Plug-and-Play Automation (for Industrial Use)